

decimals to integers. Remember that guessing the halfway point gives us the maximum possible amount of information:

```
import math
guess = math.floor(len(sorted_cabinet)/2)
```

Next, we will check whether our guess is too low or too high. We'll take a different action depending on what we find. We use the `looking_for` variable for the value we are searching for:

```
if(sorted_cabinet[guess] > looking_for):
    --snip--
if(sorted_cabinet[guess] < looking_for):
    --snip--
```

If the file in the cabinet is too high, then we'll make our guess the new upper bound, since there is no use looking any higher in the cabinet. Then our new guess will be lower—to be precise, it will be halfway between the current guess and the lower bound:

```
looking_for = 3
if(sorted_cabinet[guess] > looking_for):
    upperbound = guess
    guess = math.floor((guess + lowerbound)/2)
```

We follow an analogous process if the file in the cabinet is too low:

```
if(sorted_cabinet[guess] < looking_for):
    lowerbound = guess
    guess = math.floor((guess + upperbound)/2)
```

Finally, we can put all of these pieces together into a `binarysearch()` function. The function contains a *while* loop that will run for as long as it takes until we find the part of the cabinet we've been looking for ([Listing 4-7](#)).

```
import math
sortedcabinet = [1,2,3,4,5,6,7,8,9,10]

def binarysearch(sorted_cabinet, looking_for):
    guess = math.floor(len(sorted_cabinet)/2)
    upperbound = len(sorted_cabinet)
    lowerbound = 0
```